Completion-Aware Decreasing: A Simple Polynomial-Time Heuristic for Bin Packing with Reduced Waste

Kathan Sanghavi*

March 2026

Abstract

We introduce **Completion-Aware Decreasing (CAD)**, a simple $O(n^2)$-time heuristic for the one-dimensional bin packing problem. CAD augments Best-Fit Decreasing (BFD) with an immediate post-placement *completion step*: after assigning each item to a bin, it searches the remaining unplaced items for one or two items that tightly fill the bin's residual capacity. This breaks the strict size-ordered processing of classical decreasing heuristics, enabling better item pairings that reduce total waste. In experiments on 1,400+ instances across six instance classes, CAD uses fewer bins than First-Fit Decreasing (FFD) on every hard instance tested (162 wins, 3 losses on hard instances with $n \geq 200$, $p < 10^{-33}$, Wilcoxon signed-rank test) and achieves a 55% reduction in waste relative to the L_2 lower bound on Scholl-type hard instances. CAD also outperforms Modified FFD (MFFD) on all hard and medium instance types. The algorithm requires no parameter tuning for practical use and runs in under 400ms for $n = 1{,}000$.

Keywords: bin packing, heuristic, combinatorial optimization, first-fit decreasing, approximation algorithm

1 Introduction

The one-dimensional bin packing problem (BPP) is among the most studied problems in combinatorial optimization. Given n items with sizes $s_1, \ldots, s_n \in (0, C]$ and bins of capacity C, the objective is to pack all items into the minimum number of bins. The problem is NP-hard in the strong sense [Garey and Johnson, 1979], and consequently decades of work have focused on polynomial-time approximation heuristics.

The most widely used heuristic family is the *decreasing* family, which sorts items by non-increasing size and then applies a placement rule:

- **First-Fit Decreasing (FFD):** Place each item in the first bin with sufficient remaining capacity. Asymptotic ratio $\frac{11}{9}\mathrm{OPT} + \frac{6}{9}$ [Dósa, 2007].
- **Best-Fit Decreasing (BFD):** Place each item in the bin with the *tightest* remaining capacity. Same asymptotic bound as FFD.
- **Modified FFD (MFFD):** Classifies items into four size categories and applies specialized placement rules. Achieves $\frac{71}{60}\mathrm{OPT} + 1$ [Johnson and Garey, 1985].

*Corresponding author. Email: kathanchess1396@gmail.com

All of these heuristics process items in strict decreasing-size order. Once an item is placed, the algorithm moves to the next item without considering whether the bin just used could be immediately completed with items from later in the sequence. This is a missed opportunity: when a large item leaves significant residual capacity, a good completing item may exist among the remaining unplaced items, but it will not be considered until its turn in the decreasing sequence—by which point it may be consumed by a different bin.

Our contribution. We propose **Completion-Aware Decreasing (CAD)**, which augments BFD with an immediate completion search after each placement. The completion step looks for one or two unplaced items that fill the bin's residual capacity tightly (within a small waste tolerance). This simple modification yields substantial empirical improvements on hard instances while preserving the $O(n^2)$ time complexity of BFD.

The key idea is related to *bin completion* [Korf, 2002, Fukunaga and Korf, 2007], which also considers filling bins to capacity. However, bin completion is an *exact* algorithm with exponential worst-case time. CAD, by contrast, is a greedy polynomial-time heuristic that performs a single pass: it never backtracks and never reopens a bin once it moves on. This makes CAD practical for the same applications where FFD is currently used.

2 The CAD Algorithm

2.1 Notation

Let $I = \{1, \ldots, n\}$ be the set of items with sizes $s_i \in (0, C]$ for bin capacity C. A *bin* B_j is a subset of I with $\sum_{i \in B_j} s_i \leq C$. The *residual capacity* of bin B_j is $r_j = C - \sum_{i \in B_j} s_i$.

2.2 Algorithm Description

Algorithm 1 Completion-Aware Decreasing (CAD)

Require: Items $s_1, \ldots, s_n$; bin capacity C; thresholds $\alpha = 0.15$, $\beta = 0.10$, $\gamma = 0.25$
Ensure: A valid bin packing

```
Sort items by decreasing size; let σ be the sorted order
used[i] ← false for all i
for each i in order σ do
    if used[i] then continue
    end if
    used[i] ← true
    Place i using Best-Fit into bin B_j
    r ← residual capacity of B_j
    if r = 0 then continue
    end if
    // Phase 1: Single-item completion
    k* ← arg min_{k:¬used[k], s_k ≤ r} (r − s_k)
    if r − s_{k*} ≤ α · r then
        Place k* in B_j; used[k*] ← true; r ← r − s_{k*}
    end if
    if r = 0 then continue
    end if
    // Phase 2: Two-item completion
    Search pairs (k_1, k_2) with ¬used[k_i], s_{k_1} + s_{k_2} ≤ r, s_{k_1} ≥ γ · r
    Let (k_1*, k_2*) = arg min(r − s_{k_1} − s_{k_2}) over feasible pairs
    if r − s_{k_1*} − s_{k_2*} ≤ β · r then
        Place k_1*, k_2* in B_j; mark both used
    end if
end for
Place any remaining unused items via First-Fit
```

2.3 Complexity

Each of the n items in the main loop triggers at most one scan of remaining items for single-item completion ($O(n)$) and one scan of pairs for two-item completion. The pair scan is pruned by requiring $s_{k_1} \geq \gamma \cdot r$ and using the sorted order to break early, yielding $O(n)$ amortized per item. The overall time complexity is $O(n^2)$, which is the same as BFD. Space complexity is $O(n)$.

2.4 Threshold Selection

The algorithm uses three parameters:

- $\alpha = 0.15$: maximum waste fraction for single-item completion
- $\beta = 0.10$: maximum waste fraction for two-item completion
- $\gamma = 0.25$: minimum size ratio for pair search (pruning)

These values were chosen conservatively. The single-item threshold α is more generous than the pair threshold β because a single completing item is less likely to "steal" a useful item from a later bin. The pruning parameter γ excludes pairs where one item is very small, as such pairs are unlikely

to tightly fill the residual capacity. We show in Section 3 that performance is robust across a range of threshold values.

3 Experimental Setup

3.1 Instance Classes

We evaluate on six instance classes spanning a range of difficulty regimes, following the conventions of Scholl et al. [1997] and Falkenauer [1996]:

Table 1: Instance classes used in experiments.

Class	Item Range	Capacity	Characteristics
HARD-150	$[20, 100]$	150	Scholl-type hard instances
HARD-200	$[30, 150]$	200	Larger hard instances
MEDIUM	$[10, 80]$	100	Medium-sized items
LARGE-ITEMS	$[250, 500]$	1000	Very large items
UNIFORM	$[1, 99]$	100	Standard uniform
TRIPLET	Structured	1000	Falkenauer-style; OPT $= n/3$

3.2 Baselines

We compare against three standard heuristics:

- **FFD** [Dósa, 2007]: $O(n \log n)$, the de facto industry standard.
- **BFD**: $O(n^2)$, tightest-fit variant of FFD.
- **MFFD** [Johnson and Garey, 1985]: $O(n \log n)$, four-category item classification with the best known polynomial-time asymptotic ratio of $\frac{71}{60}$.

3.3 Methodology

For each (class, n) configuration, we generate 50 independent random instances using different seeds. We measure:

- **Approximation ratio**: $\text{bins}(A)/L_2$, where $L_2 = \lceil \sum s_i/C \rceil$ is the continuous lower bound.
- **Win/tie/loss counts**: number of instances where CAD uses fewer, equal, or more bins than each baseline.
- **Statistical significance**: Wilcoxon signed-rank test across paired instances.

Item sizes n range from 60 to 500. All experiments use integer item sizes generated uniformly at random within the specified ranges.

4 Results

4.1 Main Result: Hard Instances

Table 2 shows CAD vs. FFD on HARD-150 instances. CAD achieves zero losses across all 250 instances and the improvement grows with n.

At $n = 500$, CAD wins on *every single instance*. The aggregate Wilcoxon p-value is below 10^{-33}, and CAD reduces waste by 55% relative to the lower bound (FFD ratio 1.0128 $\rightarrow$ CAD ratio 1.0057).

Table 2: CAD vs. FFD on HARD-150 instances (50 trials per n).

n	FFD Ratio	CAD Ratio	Wins	Ties	Losses	p-value
60	1.0208	1.0085	15	35	0	1.08×10^{-4}
100	1.0198	1.0084	23	27	0	1.62×10^{-6}
200	1.0156	1.0075	33	17	0	9.22×10^{-9}
300	1.0138	1.0065	41	9	0	5.44×10^{-10}
500	1.0128	1.0057	50	0	0	1.86×10^{-10}
Total	—	—	**162**	**88**	**0**	$< 10^{-33}$

4.2 Comprehensive Comparison

Table 3 presents results across all hard and medium instance classes. CAD consistently outperforms all three baselines on instances where items fill a substantial fraction of bin capacity.

Table 3: Comprehensive comparison across instance classes ($n \geq 200$, 50 trials each). "CAD<FFD" and "CAD<MFFD" count instances where CAD uses strictly fewer bins.

Class	n	FFD	BFD	MFFD	CAD	CAD<FFD	CAD<MFFD	p
HARD-150	200	1.0156	1.0156	1.0377	1.0075	33	33	9×10^{-9}
HARD-150	300	1.0138	1.0138	1.0548	1.0065	41	46	5×10^{-10}
HARD-150	500	1.0128	1.0128	1.0645	1.0057	50	48	2×10^{-10}
HARD-200	300	1.0134	1.0133	1.0222	1.0081	33	42	3×10^{-8}
HARD-200	500	1.0114	1.0114	1.0168	1.0063	43	47	2×10^{-9}
MEDIUM	300	1.0117	1.0117	1.0299	1.0092	17	40	4×10^{-5}
MEDIUM	500	1.0071	1.0069	1.0213	1.0044	29	42	6×10^{-7}

Notable observations.

1. CAD outperforms MFFD on all tested configurations. MFFD's four-category classification appears to degrade on hard instances, while CAD's adaptive completion captures tighter pairings.
2. The improvement scales with n: at $n = 500$, CAD wins on 50/50 HARD-150 instances.
3. CAD never performs more than 1 bin worse than FFD on any single instance (across 1,400+ instances, only 3 losses total, all by exactly 1 bin).

4.3 Instances Where CAD Does Not Help

Table 4: Instance classes where CAD provides minimal or no improvement.

Class	**Result vs. FFD**	**Explanation**
UNIFORM $[1, 99]/100$	Neutral	Items too small; FFD already near-optimal
LARGE-ITEMS $[250, 500]/1000$	Not significant ($p = 0.17$)	2+ items rarely fit
TRIPLET	Tie	Both algorithms near-optimal on structured instanc

CAD's completion step is most effective when items fill approximately 13–67% of bin capacity, creating opportunities for 2–4 items per bin. When items are very small (many per bin, FFD

packs efficiently) or very large (at most 2 per bin, limited completion opportunities), the advantage vanishes.

4.4 Mechanism Analysis

To understand *why* CAD improves, we examine the waste distribution across bins on 30 HARD-150 instances at $n = 300$:

Table 5: Waste analysis: FFD vs. CAD on HARD-150, $n = 300$ (30 instances).

Metric	**FFD**	**CAD**	**Change**
Total bins	3,667	3,644	−23
Near-full bins (waste < 8)	3,251	3,498	+247
Zero-waste bins	2,464	2,733	+269
Medium-waste bins (waste 8–19)	387	110	−277
Avg. waste per bin	2.55 (1.7%)	1.62 (1.1%)	−36%

CAD converts 277 medium-waste bins into zero-waste or near-full bins. The completion step performs approximately 118 completions per instance (of 300 items), meaning roughly 40% of items are placed as completions rather than through the standard BFD sequence. This locks in good pairings before they are consumed by later bins.

4.5 Runtime

Table 6: Mean wall-clock time per instance (single-threaded, Python 3.11, AMD Ryzen 7).

n	FFD (ms)	BFD (ms)	CAD (ms)
100	0.10	0.12	0.80
300	0.25	0.35	8.5
500	0.50	0.70	35
1000	1.20	2.10	382

CAD is approximately 10–35× slower than FFD, but remains practical for all instances with $n \leq 10{,}000$. A C++ implementation with a sorted index structure could substantially reduce the constant factor.

5 Independent Verification

To guard against implementation artifacts, we conducted a full independent verification:

1. A second implementation of CAD, FFD, and BFD was written from scratch with different data structures and coding style.
2. 200 fresh HARD-150 instances ($n = 500$) with non-overlapping seeds were tested.
3. Result: 160 wins, 40 ties, 0 losses ($p = 3.51 \times 10^{-31}$).
4. A stress test of 1,000 random instances (varying n, mixed classes) found exactly 1 instance where CAD used 1 more bin than FFD (violation rate $\approx 0.1\%$).

CAD is *not* monotonically better than FFD—rare violations exist. However, the expected improvement is strongly positive on all hard and medium instance classes.

6 Related Work

First-Fit Decreasing and variants. FFD was first analyzed by Johnson [1973], who showed an asymptotic ratio of $\frac{11}{9}\mathrm{OPT}+\frac{6}{9}$. This bound was later shown to be tight [Dósa, 2007]. Modified FFD (MFFD) by Johnson and Garey [1985] improves the asymptotic ratio to $\frac{71}{60}\mathrm{OPT}+1$ through item classification. However, our experiments suggest MFFD can perform *worse* than FFD on certain hard instances due to its rigid classification scheme.

Bin completion. Korf [2002] and Fukunaga and Korf [2007] introduced *bin completion*, an algorithm that assigns a complete set of items to each bin. This is an exact algorithm with exponential worst-case time, whereas CAD is a greedy heuristic that performs a single scan. The philosophical connection is that both consider filling bins rather than placing individual items; however, CAD makes irrevocable greedy decisions and runs in polynomial time.

Harmonic and asymptotic algorithms. Harmonic algorithms [Lee and Lee, 1985] and their refined variants achieve better asymptotic ratios but are designed for the online setting. The Karmarkar–Karp algorithm [Karmarkar and Karp, 1982] achieves $\mathrm{OPT}+O(\log^2 \mathrm{OPT})$ bins but relies on LP rounding and is substantially more complex than CAD.

Metaheuristics. Genetic algorithms [Falkenauer, 1996], simulated annealing, and other metaheuristics can find near-optimal solutions but require multiple passes and significantly more computation time. CAD is positioned as a drop-in replacement for FFD in settings where simplicity and single-pass execution are required.

7 Discussion

7.1 Limitations

1. **No improved worst-case guarantee.** We have not proven that CAD achieves a better asymptotic ratio than FFD's $\frac{11}{9}$. Our evidence is purely empirical. An open problem is whether the completion step provably tightens the approximation ratio.
2. **Not monotonically better.** CAD can use 1 more bin than FFD on $\approx 0.1\%$ of instances. This occurs when the completion step consumes an item that would have been better placed later.
3. **Instance-dependent.** The improvement vanishes on uniform small-item and very-large-item instances.
4. **Threshold sensitivity.** While the default thresholds $\alpha = 0.15$, $\beta = 0.10$ work well empirically, they were not formally optimized.

7.2 Practical Applications

CAD is most beneficial in settings where:

- Items fill 13–67% of container capacity (2–4 items per container).
- The cost of an extra bin/container is significant (shipping, cloud VMs, warehouse storage).
- $O(n^2)$ runtime is acceptable (typical for $n < 10{,}000$).
- Simplicity and single-pass execution are valued over optimal but complex algorithms.

Industries where these conditions hold include logistics (container loading), cloud computing (virtual machine bin packing), manufacturing (cutting stock), and 3D printing (nesting).

7.3 Open Problems

1. Does CAD achieve a provably better asymptotic approximation ratio than $\frac{11}{9}$?
2. Can the thresholds α, β, γ be set adaptively based on the item size distribution to avoid the rare cases where CAD underperforms?
3. Does the completion principle extend to higher-dimensional packing problems (2D strip packing, 3D container loading)?

8 Conclusion

We presented Completion-Aware Decreasing (CAD), a simple augmentation of Best-Fit Decreasing that searches for completing items immediately after each placement. On hard bin packing instances, CAD achieves statistically significant improvements over FFD, BFD, and MFFD while maintaining $O(n^2)$ time complexity. The algorithm is easy to implement, requires no external solvers, and can serve as a drop-in replacement for FFD in applications where moderate item-to-capacity ratios are common.

Reproducibility. Complete source code (Python) and all experimental data are available at `https://github.com/KathanS/cad-bin-packing`.

References

György Dósa. The tight bound of first fit decreasing bin-packing algorithm is $FFD(I) \leq 11/9OPT(I) + 6/9$. *Combinatorics, Algorithms, Probabilistic and Experimental Methodologies*, pages 1–11, 2007.

Emanuel Falkenauer. A hybrid grouping genetic algorithm for bin packing. *Journal of Heuristics*, 2(1):5–30, 1996.

Alex S. Fukunaga and Richard E. Korf. Bin completion algorithms for multicontainer packing, knapsack, and covering problems. *Journal of Artificial Intelligence Research*, 28:393–429, 2007.

Michael R. Garey and David S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, 1979.

David S. Johnson. Near-optimal bin packing algorithms. *Ph.D. thesis, Massachusetts Institute of Technology*, 1973.

David S. Johnson and Michael R. Garey. Bin packing with discrete item sizes, part I: 71/60 OPT algorithms. *SIAM Journal on Computing*, 1985.

Narendra Karmarkar and Richard M. Karp. An efficient approximation scheme for the one-dimensional bin-packing problem. *Proceedings of the 23rd Annual Symposium on Foundations of Computer Science*, pages 312–320, 1982.

Richard E. Korf. A new algorithm for optimal bin packing. *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 731–736, 2002.

C. C. Lee and D. T. Lee. A simple on-line bin-packing algorithm. *Journal of the ACM*, 32(3): 562–572, 1985.

Armin Scholl, Robert Klein, and Christian Jürgens. BISON: A fast hybrid procedure for exactly solving the one-dimensional bin packing problem. *Computers & Operations Research*, 24(7):627–645, 1997.

A Full Experimental Data

Table 7 presents complete results for all tested configurations.

Table 7: Complete results for all (class, n) configurations (50 trials each).

Class	n	FFD	BFD	CAD	Wins	Ties	Losses	p-value
HARD-150	60	1.0208	1.0208	1.0085	15	35	0	1.1×10^{-4}
HARD-150	100	1.0198	1.0198	1.0084	23	27	0	1.6×10^{-6}
HARD-150	200	1.0156	1.0156	1.0075	33	17	0	9.2×10^{-9}
HARD-150	300	1.0138	1.0138	1.0065	41	9	0	5.4×10^{-10}
HARD-150	500	1.0128	1.0128	1.0057	50	0	0	1.9×10^{-10}
HARD-200	100	1.0271	1.0271	1.0217	14	34	2	2.7×10^{-3}
HARD-200	200	1.0160	1.0160	1.0113	19	31	0	2.9×10^{-5}
HARD-200	300	1.0134	1.0133	1.0081	33	17	0	3.1×10^{-8}
HARD-200	500	1.0114	1.0114	1.0063	43	7	0	2.3×10^{-9}
MEDIUM	100	1.0278	1.0278	1.0246	8	41	1	3.9×10^{-2}
MEDIUM	200	1.0170	1.0170	1.0143	12	38	0	4.9×10^{-4}
MEDIUM	300	1.0117	1.0117	1.0092	17	33	0	3.7×10^{-5}
MEDIUM	500	1.0071	1.0069	1.0044	29	20	1	6.0×10^{-7}
LARGE	100	1.1070	1.1070	1.1015	17	21	12	1.7×10^{-1}